

Урок 7. Корутины. Scope

24 декабря 2019

В этом уроке рассмотрим, что такое Scope и как создать его самый простой вариант.

Пора нам уже переходить к практике и начинать делать примеры. Но перед этим надо познакомиться с объектом, без которого корутину не запустить. Это Scope.

Lifecycle

Для начала давайте рассмотрим, как scope поможет нам ограничить корутины.

Когда мы запускаем корутину, она живет своей жизнью. Она будет продолжать свою работу, пока не закончит, даже если результаты этой работы уже не нужны (пользователь отменил операцию, закрыл экран и т.п.).

Чтобы избежать этой ненужной работы, мы можем получать job от билдера:

```
1 | val job = launch {  
2 |     // coroutine code  
3 | }
```

и использовать его метод cancel.

```
1 | job.cancel()
```

Тем самым мы отменим корутину, если она более не нужна, но продолжает выполняться.

О том, что такое отмена корутины и как ее делать правильно, мы еще поговорим в отдельном уроке. Пока же просто скажу, что механизм примерно тот же, что и при отмене AsyncTask. Т.е. нам самим надо будет проверять статус.

Если корутин много или они вложенные, то становится неудобно собирать все их джобы и потом отменять их. Scope помогает решить эту проблему.

Используем scope для запуска корутин:

```
1 | scope.launch {  
2 |     // coroutine code  
3 | }
```

Используя один scope мы можем запустить несколько корутин. А когда нам понадобится все эти корутины отменить, мы просто отменяем scope:

```
1 | scope.cancel()
```

Этот метод cancel можно вызывать, например, в Activity.onDestroy(). Тогда при закрытии экрана все корутины будут отменены.

Без Scope никуда

Т.е. scope - это полезный инструмент, который удобно использовать, чтобы не было бесхозных корутин в приложении. Более того, этот инструмент настолько полезный, что является обязательным при создании корутин. Без него корутину не создать.

Когда в примере первого урока я показывал простой пример корутины с билдером launch, я говорил, что такой код работать не будет и нужны дополнительные действия. Если вы попытаете в любом месте вашего кода вызвать launch, то у вас ничего не получится. Давайте посмотрим почему.

Обратим внимание на исходники билдера launch:

```
1 | public fun CoroutineScope.launch
```

Видим, что launch - это расширение класса CoroutineScope, который и является scope объектом. Без этого объекта мы не сможем вызвать билдер корутины.

Давайте подробно посмотрим, что такое CoroutineScope:

```
1 | public interface CoroutineScope {  
2 |     // By convention, should contain an instance of a [Job][Job] to enforce structured concurrency.  
3 |     public val coroutineContext: CoroutineContext  
4 | }
```

Это интерфейс. Объект, реализующий этот интерфейс, содержит CoroutineContext. Мы пока не изучали, что это. Если вкратце - то это хранилище, типа Map, в котором могут храниться различные объекты. Об этом мы еще поговорим в дальнейших уроках.

Обратите внимание на комментарий к полю coroutineContext. По контракту контекст скопа хранит в себе Job скопа. Этот Job связан с корутинами, создаваемыми в этом scope. Когда для scope вызывается билдер, который создает корутину, Job создаваемой корутины подписывается на Job из scope. Вызывая scope.cancel, мы отменяем Job в scope, а он отменит все подписанные на него Job-ы, т.е. все корутины.

Откуда взять scope?

Scope может быть предоставлен некоторыми объектами с жизненным циклом. Если мы используем этот scope, чтобы запускать корутины, то все его корутины будут отменены, когда объект завершает свою работу.

Пример такого объекта - [ViewModel](#) из Architecture Components. У него есть свойство [viewModelScope](#).

```
1 class MyViewModel: ViewModel() {
2     init {
3         viewModelScope.launch {
4             // Coroutine that will be canceled when the ViewModel is cleared.
5         }
6     }
7 }
```

Корутины, запущенные в этом scope, будут отменены, когда завершится ViewModel.

Аналогично есть [scope](#) и для [Lifecycle](#) объектов.

Создание Scope

Scope объект можно создать самостоятельно. Мы будем определять, когда следует отменять его и все его корутины. Давайте создадим такой пример.

Для этого в проект подключите корутины:

```
1 implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-core:1.3.0'
2 implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-android:1.3.0'
```

Самый простой scope можно создать так:

```
1 val scope = CoroutineScope(Job())
```

Под капотом используется самая простая реализация интерфейса CoroutineScope. В ее Context помещается созданный нами Job.

Теперь можно запускать корутины, используя этот scope:

```
1 scope.launch {
2     Log.d(TAG, "first coroutine")
3 }
```

Эта корутина хоть и бессмысленна, но для простого примера этого достаточно. В последующих уроках будем добавлять функционал, разбираться с отменой, завершением, потоками и т.д. И scope создавать будем посложнее.

Для одного и того же скопа можно создавать несколько корутин:

```
1 scope.launch {
2     Log.d(TAG, "first coroutine")
3 }
4
5 scope.launch {
6     Log.d(TAG, "second coroutine")
7 }
8
9 scope.launch {
10    Log.d(TAG, "third coroutine")
11 }
```

Эти корутины только выводят лог и сами завершаются очень быстро. Но если бы там выполнялась долгая работа, то scope помог бы нам ограничить время жизни этих корутин. Например, мы решаем, что при закрытии экрана все корутины должны отмениться. Для этого помещаем вызов scope.cancel в метод Activity.onDestroy

```
1 override fun onDestroy() {
2     super.onDestroy()
3     scope.cancel()
4 }
```

Если, кстати, заглянуть в исходники отмены scope, то там увидим следующее:

```
1 public fun CoroutineScope.cancel(cause: CancellationException? = null) {
2     val job = coroutineContext[Job] ?: error("Scope cannot be cancelled because it does not have a job: $this")
3     job.cancel(cause)
4 }
```

Из контекста берется Job скопа и отменяется. Это приводит к тому, что отменяются все подписанные на него Job-ы корутин.

Соответственно, если не вызвать scope.cancel, то корутины этого scope не отменяются. Поэтому не теряйте этот вызов, если вам необходимо отменять корутины.

Вложенные корутины

Давайте рассмотрим, как работает scope при создании дочерней корутины внутри родительской. Т.е. вызова launch внутри launch.

У нас есть билдер, который создает родительскую корутину.

```
1 | scope.launch {
2 |     // coroutine code block
3 | }
```

Нас сейчас интересует то, что находится между фигурными скобками - т.е. блок кода корутины.

Давайте посмотрим на описание launch

```
1 | public fun CoroutineScope.launch(
2 |     context: CoroutineContext = EmptyCoroutineContext,
3 |     start: CoroutineStart = CoroutineStart.DEFAULT,
4 |     block: suspend CoroutineScope.() -> Unit
5 | )
```

Блок кода имеет тип CoroutineScope.() -> Unit. Это значит, что в этом блоке кода нам доступен this с типом CoroutineScope.

В Android Studio это видно:



```
scope.launch { this: CoroutineScope
}
}
```

Мы можем использовать этот this, как scope, для вызова билдера корутины:

```
1 | scope.launch {
2 |     // parent coroutine code block
3 |     this.launch {
4 |         // child coroutine code block
5 |     }
6 | }
```

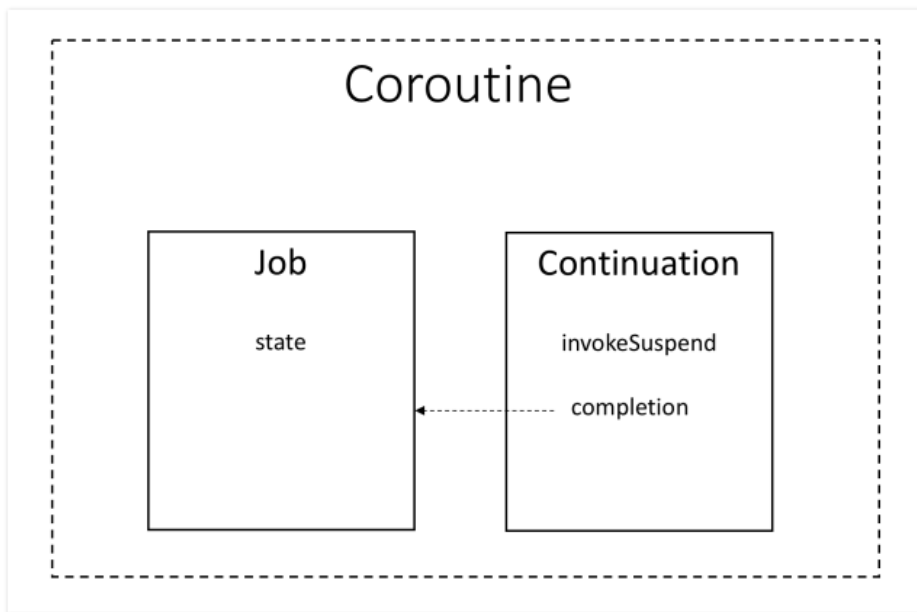
или можно без явного использования this:

```
1 | scope.launch {
2 |     // parent coroutine code block
3 |     launch {
4 |         // child coroutine code block
5 |     }
6 | }
```

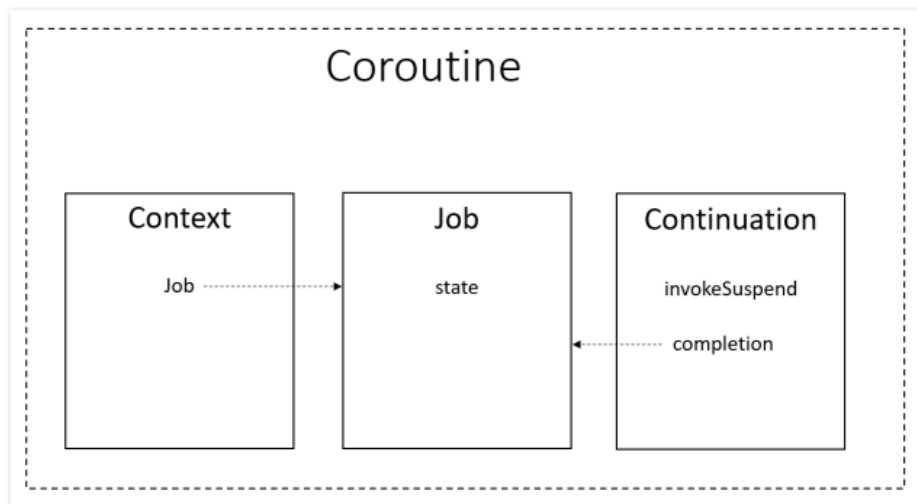
У scope, который доступен нам в блоке кода родительской корутины, должен быть Job. Это Job родительской корутины. Job дочерней корутины подписывается на него. Таким образом получается, что родительская корутина - это scope для дочерней. И если отменить родительскую корутину, то отменится и дочерняя. Об этом мы еще подробно поговорим в последующих уроках.

Давайте добавим новые объекты на схему корутины.

Было так:



Стало так:



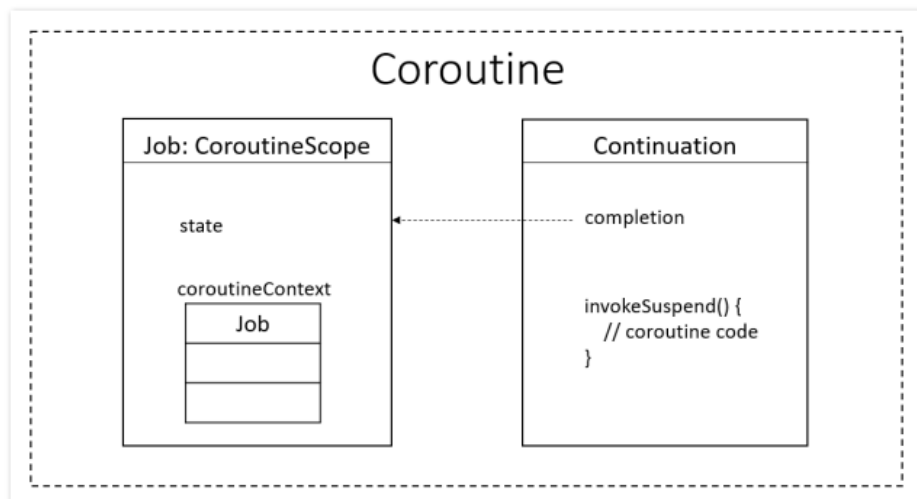
Добавляем контекст, в котором могут находиться различные объекты. Как минимум там хранится Job корутины.

Обратите внимание, на схеме нет ничего про CoroutineScope. Хотя именно в CoroutineScope должен храниться Context. Также этот CoroutineScope объект (в виде this) мы получаем в блоке кода корутины (внутри launch).

На схеме нет отдельного CoroutineScope объекта, потому что его роль исполняет Job.

Я в прошлом уроке говорил, что в исходниках корутин у класса Job есть наследники, которые используются билдером в качестве Job для создаваемых корутин. И вот эти наследники реализуют интерфейс CoroutineScope. А значит у них есть Context. И в этом контексте они хранят самих себя, как Job.

Еще немного изменим схему корутины:



Job наследует интерфейс CoroutineScope и хранит ссылку на Context. Т.к. Context должен содержать Job, то Job просто помещает в Context ссылку на себя.

Получается, что Job является scope, и сам же выступает в качестве Job этого scope.

Чтобы это проверить, сделайте такой пример:

```
1 | val job = scope.launch {  
2 |     Log.d(TAG, "scope = $this")  
3 | }  
4 | Log.d(TAG, "job = $job")
```

Вы увидите, что job, возвращаемый билдером, и scope в его корутине - это один и тот же объект.